

Evaluation of obsolete modeling functions

Gabriel Arellano

2026-09-01

Contents

1	Goal	1
2	Candidate functions for removal	2
	lmerBayes framework	3
	modelBayes framework	4
	Generic hand-written MCMC functions	4
	Paper-specific demographic abundance model	4
	Mortality model functions	5
	Growth-bin modeling	5
	Growth-bin model graphics	5
2.1	Earlier CTFS evaluation	6
2.2	Earlier tests (ctfs+fgeo development)	6
3	Whole-package dependencies before work	6
3.1	Read code and declare parameters	6
3.2	Make the dependency graph	8
3.3	Dependency map	10
4	Detailed evaluation and decisions	13
4.1	Evaluation placeholder	13
5	Whole-package dependencies after work	13
5.1	Post-work placeholder	13

1 Goal

The goal is to make the fgeo2 body of code lighter and easier to maintain by removing obsolete modeling functions. fgeo2 will focus on metrics, descriptors, and other reusable analytical

primitives. Statistical modeling requires flexibility and is better handled with maintained, dedicated tools rather than project-specific model-fitting and MCMC infrastructure. This evaluation focuses on:

- primarily, removal of modeling functions;
- minimum edits to the remaining ones; and
- identifying dependents somewhere else, so all code remains working.

2 Candidate functions for removal

The initial scope includes hand-written Bayesian and MCMC infrastructure, specialized demographic and growth models, and functions dedicated to fitting or displaying those models.

```
# Candidate means "evaluate for removal", not "approved for deletion".
candidate_groups <- list(
  `lmerBayes framework` = c(
    "lmerBayes", "lmerBayes.hyperllike.sigma",
    "lmerBayes.hyperllike.mean", "full.likelihood.lmerBayes",
    "llike.fixef.lmer", "llike.model.lmer",
    "residual.llike.lmerBayes", "badSD", "arrangeParam.llike.2D",
    "arrangeParam.Gibbs.2D", "saveParamFile", "restoreParamFile",
    "resetParam", "summaryMCMC", "recalculate.lmerBayesllike",
    "covTocorr"
  ),
  `modelBayes framework` = c(
    "modelBayes", "residual.llike.modelBayes", "summaryModelMCMC"
  ),
  `Generic hand-written MCMC functions` = c(
    "regression.Bayes", "Gibbs.regslope", "Gibbs.regsigma",
    "Gibbs.normalmean", "Gibbs.normalvar", "model.xy",
    "arrangeParam.llike", "arrangeParam.Gibbs", "llike.GaussModel",
    "llike.GaussModelSD", "BadParam", "graph.modeldiag",
    "metrop1step", "metrop1step.discrete", "testmcmcfunc"
  ),
  `Paper-specific demographic abundance model` = c(
    "model.littleR.Gibbs", "full.abundmodel.llike", "prob.N1",
    "spmean.mort.abundGibbs", "hyper.abundGibbs", "hyper.mortGibbs",
    "bad.asympower.param", "bad.asymexp.param", "fitSeveralAbundModel",
    "graph.abundmodel", "find.xaxis.hist", "abundmodel.fit"
  ),
  `Mortality model functions` = c(
```

```

    "lmerMortLinear", "lmerMortFixedTime"
  ),
  `Growth-bin modeling` = c(
    "extract.growthdata", "run.growthbin.manyspp", "run.growthfit.bin",
    "growth.flexbin", "llike.linearbin.optim", "defineBinBreaks",
    "defineSDpar", "enoughSamplePerBin", "wideEnoughBins",
    "bad.binparam", "bad.binsdpar", "calculateBinModel.BIC",
    "calculateBinModel.DIC", "calculateBinModel.AIC",
    "calculateBinModel.bestpred", "assembleBinOutput"
  ),
  `Growth-bin model graphics` = c(
    "graph.growthmodel.spp", "graph.growthmodel",
    "overlay.growthbinmodel", "compare.growthbinmodel",
    "graph.outliers.spp", "graph.outliers", "binGraphSampleSpecies",
    "binGraphManySpecies.Panel", "binGraphManySpecies"
  )
)

candidate_functions <- unique(unlist(candidate_groups, use.names = FALSE))

for(group in names(candidate_groups))
{
  cat("### ", group, "\n\n", sep = "")
  cat(paste0("-  ", candidate_groups[[group]], "()", collapse = "\n"))
  cat("\n\n")
}

```

lmerBayes framework

- lmerBayes()
- lmerBayes.hyperllike.sigma()
- lmerBayes.hyperllike.mean()
- full.likelihood.lmerBayes()
- llike.fixef.lmer()
- llike.model.lmer()
- residual.llike.lmerBayes()
- badSD()
- arrangeParam.llike.2D()
- arrangeParam.Gibbs.2D()
- saveParamFile()
- restoreParamFile()

- `resetParam()`
- `summaryMCMC()`
- `recalculate.lmerBayesllike()`
- `covTocorr()`

modelBayes framework

- `modelBayes()`
- `residual.llike.modelBayes()`
- `summaryModelMCMC()`

Generic hand-written MCMC functions

- `regression.Bayes()`
- `Gibbs.regslope()`
- `Gibbs.regsigma()`
- `Gibbs.normalmean()`
- `Gibbs.normalvar()`
- `model.xy()`
- `arrangeParam.llike()`
- `arrangeParam.Gibbs()`
- `llike.GaussModel()`
- `llike.GaussModelSD()`
- `BadParam()`
- `graph.modeldiag()`
- `metrop1step()`
- `metrop1step.discrete()`
- `testmcmcfunc()`

Paper-specific demographic abundance model

- `model.littleR.Gibbs()`
- `full.abundmodel.llike()`
- `prob.N1()`
- `spmean.mort.abundGibbs()`
- `hyper.abundGibbs()`
- `hyper.mortGibbs()`
- `bad.asympower.param()`
- `bad.asymexp.param()`
- `fitSeveralAbundModel()`
- `graph.abundmodel()`

- `find.xaxis.hist()`
- `abundmodel.fit()`

Mortality model functions

- `lmerMortLinear()`
- `lmerMortFixedTime()`

Growth-bin modeling

- `extract.growthdata()`
- `run.growthbin.manyspp()`
- `run.growthfit.bin()`
- `growth.flexbin()`
- `llike.linearbin.optim()`
- `defineBinBreaks()`
- `defineSDpar()`
- `enoughSamplePerBin()`
- `wideEnoughBins()`
- `bad.binparam()`
- `bad.binsdpar()`
- `calculateBinModel.BIC()`
- `calculateBinModel.DIC()`
- `calculateBinModel.AIC()`
- `calculateBinModel.bestpred()`
- `assembleBinOutput()`

Growth-bin model graphics

- `graph.growthmodel.spp()`
- `graph.growthmodel()`
- `overlay.growthbinmodel()`
- `compare.growthbinmodel()`
- `graph.outliers.spp()`
- `graph.outliers()`
- `binGraphSampleSpecies()`
- `binGraphManySpecies.Panel()`
- `binGraphManySpecies()`

2.1 Earlier CTFS evaluation

The earlier [CTFS function-decision table](#) was edited by Sean McMahon and Gabriel Arellano and discussed with Stuart Davies and Mauro Lepore. It records the following judgments:

- deprecate the complete `lmerBayes()` and `modelBayes()` blocks;
- deprecate the demographic abundance model because it was developed for one paper;
- try to deprecate the growth-bin modeling block;
- classify `lmerMortLinear()` and `lmerMortFixedTime()` as low-priority model functions;
- treat much of `utilities-statistics.r` as overly specific, while recognizing that its functions require individual review because some may have broader uses. How useful these functions are for other uses requires evaluation of the downstream dependents.

2.2 Earlier tests (ctfs+fgeo development)

- The [CTFS development notes](#) record that `modelBayes()` still entered debugging and failed after an attempted maintenance fix.
- `growth.flexbin()`, `run.growthfit.bin()`, and `run.growthbin.manyspp()` failed in the growth tutorial. Their attempted calls remain commented out in [test-growth_vs_dbh.R](#).
- The `lmerBayes()` regression test was disabled because its required data were no longer available. Even when enabled, it checked only the names of the returned components, not the fitted result; see [test-replace_subset.R](#).
- The development notes report that one test of `model.littleR.Gibbs()` and `fitSeveralAbundModel()` took approximately 20 minutes. No active automated test for that model remains in the CTFS test suite.

3 Whole-package dependencies before work

3.1 Read code and declare parameters

The dependency structure below uses every R script in the `R/new` directory of all seven `fgeo2` modules. The exact pre-work snapshot is commit [040b73e](#). Git reads the files directly from that commit, so later changes in the working tree are not mixed into the evaluation.

```
path <- normalizePath(system2("git", c("rev-parse", "--show-toplevel"),
                                stdout = TRUE)[1])

library(igraph, quietly = TRUE)
```

Warning: package 'igraph' was built under R version 4.4.3

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

```
source(file.path(path, "evaluation/0-code-to-source.r"))

pre_work_commit <- "040b73e0cd9a0d55d64c1639b22159d5c6cbd20e"
pre_work_date <- as.Date("2026-09-01")

modules <- c(
  "01_utilities", "02_biomass", "03_spatial",
  "04_abundance_diversity", "05_demography_growth",
  "06_topography_habitat", "07_mapping_plotting"
)

all_files <- system2(
  "git",
  c("-C", shQuote(path), "ls-tree", "-r", "--name-only", pre_work_commit),
  stdout = TRUE
)

module_pattern <- paste(modules, collapse = "|")
scripts <- all_files[
  grepl(paste0("^(", module_pattern, ")/R/new/.*\.[Rr]?$"), all_files)
]

code_env <- new.env(parent = .GlobalEnv)

for(file in scripts)
{
  code <- system2(
    "git",
```

```

    c("-C", shQuote(path), "show", paste0(pre_work_commit, ":", file)),
    stdout = TRUE
  )

  eval(parse(text = paste(code, collapse = "\n")), envir = code_env)
}

objects <- mget(
  ls(code_env, all.names = TRUE),
  envir = code_env,
  inherits = FALSE
)

all_functions <- names(objects)[vapply(objects, is.function, logical(1))]

edges <- get_all_dependencies(envir = code_env)
edges <- edges[, c("from_function", "to_function"), drop = FALSE]
colnames(edges) <- c("from", "to")

candidates <- intersect(candidate_functions, all_functions)
missing_candidates <- setdiff(candidate_functions, all_functions)

if(length(missing_candidates))
  stop("Candidate functions not found: ", paste(missing_candidates, collapse = ", "))

```

3.2 Make the dependency graph

All functions found in the seven R/new directories are passed to the graph as vertices. Consequently, the graph retains functions with no dependency edges as isolated vertices rather than omitting them.

```

g.before <- graph_from_data_frame(edges, directed = TRUE)

downstream <- sapply(candidates, function(candidate) downstream_dependents(candidate, edges))
downstream <- setdiff(unique(downstream), candidates)

V(g.before)$color <- "gray80"
V(g.before)$color[V(g.before)$name %in% downstream] <- "yellow"
V(g.before)$color[V(g.before)$name %in% candidates] <- "orange"

pre_work_pdf <- file.path(

```



```

path,
"evaluation/results",
paste0(
  "fgeo2-dependencies-", substr(pre_work_commit, 1, 7), "-",
  format(pre_work_date, "%Y-%m-%d"), ".pdf"
)
)

dir.create(dirname(pre_work_pdf), recursive = TRUE, showWarnings = FALSE)

set.seed(1)

# Control the overall canvas with height and width.
pdf(pre_work_pdf, height = 33, width = 33)
par(mar = c(0, 0, 2, 0))

plot(
  g.before,
  layout = layout.fruchterman.reingold,
  vertex.label = V(g.before)$name,
  vertex.size = 6,          # node size
  vertex.color = V(g.before)$color,
  vertex.frame.color = "white",
  vertex.label.color = "black",
  vertex.label.cex = 2,     # label size
  edge.color = "gray45",
  edge.width = 0.7,         # line width
  edge.arrow.size = 0.3     # arrow size
)

legend(
  "topleft",
  legend = c(
    "Candidate for removal",
    "Downstream dependent",
    "Other function"
  ),
  pt.bg = c("orange", "yellow", "gray80"),
  pch = 21,
  pt.cex = 2,
  bty = "n"
)

```

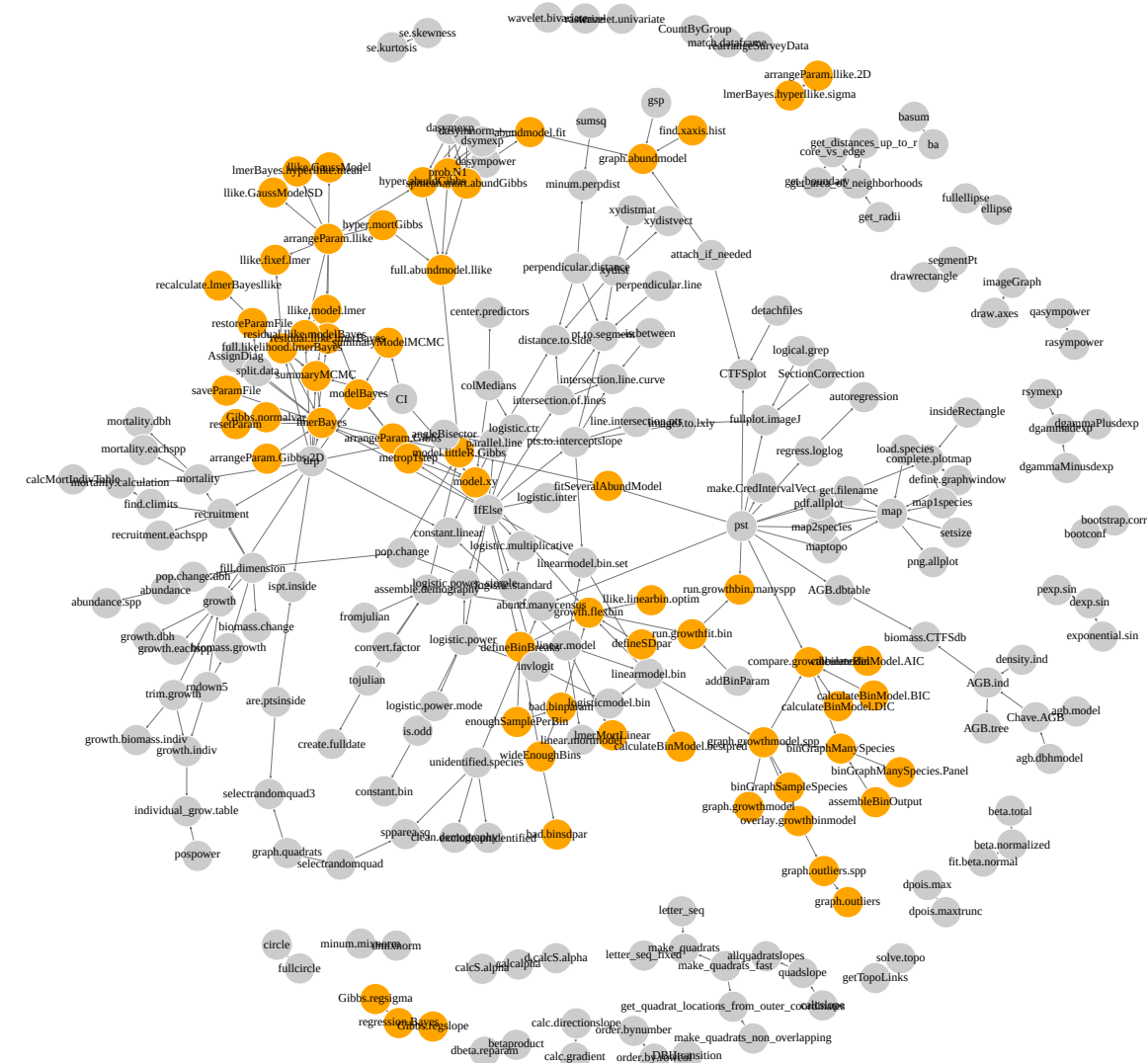
```
dev.off()
```

pdf
2

3.3 Dependency map

Arrows point from a dependency to the function that uses it. Candidate functions are orange. Their downstream dependents, followed through any number of dependency levels, are yellow. All other functions are gray. Every function found under the seven **R/new** directories is present and labeled, including isolated functions.

● candidate for removal
● non-candidate dependent
● other function



The same graph is stored separately as `evaluation/results/fgeo2-dependencies-040b73e-2026-09-01.pdf` so it can be inspected at full size.

```

candidate_dependencies <- unique(edges$from[
  !(edges$from %in% candidates) &
  edges$to %in% candidates
])

dependency_summary <- data.frame(

```

```

Measure = c(
  "R/new scripts",
  "Functions",
  "Functions shown in the dependency graph",
  "Inter-function dependencies",
  "Candidate functions",
  "Dependencies among candidate functions",
  "Other functions used directly by candidates",
  "Other functions depending on candidates at any level"
),
Value = c(
  length(scripts),
  length(all_functions),
  length(V(g.before)),
  nrow(edges),
  length(candidates),
  sum(
    edges$from %in% candidates &
    edges$to %in% candidates
  ),
  length(candidate_dependencies),
  length(downstream)
)
)

knitr::kable(dependency_summary, row.names = FALSE)

```

Measure	Value
R/new scripts	26
Functions	357
Functions shown in the dependency graph	239
Inter-function dependencies	289
Candidate functions	73
Dependencies among candidate functions	67
Other functions used directly by candidates	18
Other functions depending on candidates at any level	16

4 Detailed evaluation and decisions

This section is intentionally left open for the function-by-function or cluster-by-cluster evaluation. For each candidate, the evaluation should record at least:

- its analytical purpose;
- current dependencies and dependents;
- available tests and known failures;
- whether it is a metric/descriptor or part of model fitting;
- the maintained external tools that cover the modeling task, when relevant;
- the decision to retain, replace, adapt, or remove it, with concise reasoning;
- any downstream changes required before removal.

4.1 Evaluation placeholder

To be completed before any functions are removed.

5 Whole-package dependencies after work

This section is intentionally left as a placeholder. After the removals and any necessary changes to dependents are complete, it will record the post-work commit and date, reproduce the whole-package dependency graph, and compare the resulting structure with the pre-work snapshot.

5.1 Post-work placeholder

To be completed after the work.